

■ 1.7.10 Advanced Topic: Rearranging Nested Lists

You will encounter nested lists if you use matrices, or generate multidimensional arrays and tables. Rearranging nested lists can be a complicated affair, and you will often have to experiment to get the right combination of commands.

<code>Transpose[list]</code>	interchange the top two levels of lists
<code>Transpose[list, n]</code>	interchange the top level with the n^{th} level
<code>Flatten[list]</code>	flatten out all levels in <i>list</i>
<code>Flatten[list, n]</code>	flatten out the top <i>n</i> levels in <i>list</i>

Functions for rearranging nested lists.

Here is a 3×2 array.

```
In[1]:= t = {{a, b}, {c, d}, {e, f}}
Out[1]= {{a, b}, {c, d}, {e, f}}
```

You can rearrange it to get a 2×3 array.

```
In[2]:= Transpose[t]
Out[2]= {{a, c, e}, {b, d, f}}
```

This “flattens out” sublists. You can think of it as effectively just removing the inner sets of braces.

```
In[3]:= Flatten[t]
Out[3]= {a, b, c, d, e, f}
```

Here is a $2 \times 2 \times 2$ array.

```
In[4]:= t = Table[i^2 + j^2 + k^2, {i, 2}, {j, 2}, {k, 2}]
Out[4]= {{{3, 6}, {6, 9}}, {{6, 9}, {9, 12}}}
```

This flattens out all the levels.

```
In[5]:= Flatten[t]
Out[5]= {3, 6, 6, 9, 6, 9, 9, 12}
```

This flattens only the first level of sublists.

```
In[6]:= Flatten[t, 1]
Out[6]= {{3, 6}, {6, 9}, {6, 9}, {9, 12}}
```

There are many other operations you can perform on nested lists. We will discuss some more of them when we look at `Map`, `Apply`, `Scan` and `Level` in Section 2.1.